SAVAGE WRESTLING ACADEMY

Platform project handoff

Source-verified technical review · November 5, 2026

Prepared for an experienced AI and business-systems builder to inspect, assess and continue development.

Sites version 70 · deployed source bf444860637d

252 tracked files · 13 migrations · 15 business tables

18 existing test scripts passed · TypeScript and build passed

Application and hosted data unchanged during review

Key decisions before production

Complete billing and enrollment infrastructure, resolve authentication-origin and dependency findings, and establish complete recoverable backups. Much of the operational owner workspace still uses sample data.

This report distinguishes implemented code from live verification. A passing isolated test is not evidence that production email, payments or a customer workflow succeeded.

Reading guide

1. Savage Wrestling Academy platform handoff
2. Architecture
3. Feature inventory
4. Codebase overview
5. User workflows
6. Database schema and migrations
7. Route reference
8. Setup and continued development
9. Version control and provenance
10. Known issues and technical debt
11. Prioritized development roadmap
12. Demo and access
13. Verification record and limits
14. Access required for collaboration
15. Outstanding tasks

The accompanying ZIP contains this PDF, the exact source snapshot, editable Markdown documentation, an environment template, schema exports, screenshots and verification evidence. The archived source retains its original documentation unchanged.

Savage Wrestling Academy platform handoff

Prepared November 5, 2026 for a technical collaborator. This package is a source-verified review of the existing platform. It includes a complete tracked-source snapshot and new handoff documentation. The original application, accounts, database contents, and deployment were not changed during this review.

Start here

1. Read `SWA_Project_Handoff_2026-11-05.pdf` for the complete review.
2. Read `docs/ARCHITECTURE.md` and `docs/FEATURE_STATUS.md` before estimating work.
3. Follow `docs/SETUP.md` in a separate development environment.
4. Review `docs/ISSUES.md`, `docs/ROADMAP.md`, and the dependency audit before enabling payments.
5. Use the public demos in `docs/DEMO_ACCESS.md` for initial evaluation.

Current state

The platform combines working authenticated backend features with an extensive sample-data admin workspace. Public calendars, family profiles, coach openings, program approvals, approval chat, practice plans, curricula, and scholarships have implemented backend paths and isolated tests. Paid registration is a review flow. It cannot currently collect money or activate a paid membership. Camera attendance recognizes sample passes and does not record real QR attendance.

The admin workspace's sample wrestlers are stored separately from real family-account wrestlers. A sample membership, check-in, program transfer, message, or lesson payout must not be treated as a completed real-world transaction.

Snapshot identity

Item	Verified value
Source commit	`bf444860637d6fc94fbd0342bf7ba0963ed0f178`
Sites version	70
Live deployment	Succeeded; same source commit
Tracked source	252 files, 2,433,614 bytes
Local Git history	73 commits; not shallow
Configured Git remotes	None in inspected checkout
GitHub mirror	Not verified
Validation	All 18 existing test scripts passed; typecheck and build passed
Dependency audit	8 advisories, including 2 critical; runtime reachability needs assessment

Package contents

- `source/` contains the exact tracked files from the reviewed commit, including the original README and security document. Their stale statements are called out in this review.

- `docs/` contains the updated project README-equivalent documentation, architecture, feature matrix, API reference, workflows, setup, schema, issues, roadmap, and access instructions.

- `.env.example` contains placeholders only. See the setup document before configuring local authentication.

- `evidence/` contains test results, build/typecheck logs, dependency audit, and sanitized service metadata.

- `docs/schema.sql` and `docs/schema.json` were derived by applying all 13 tracked migrations to an empty isolated SQLite database.

- `docs/source-manifest.json` identifies every source file and SHA-256 hash.

- `docs/reproductions/sample-event-reference-error.mjs` reproduces a confirmed sample camp/event review bug without accessing production.

Secrets, customer records, database dumps, authentication sessions, `node_modules`, generated build output, and Git object history are excluded. A source snapshot does not include the operational database or hosted secret values. Request account-specific access from Justin through the appropriate service.

Architecture

Deployment and application shape

The live origin is `https://savage-registration-preview.justinaccordino546.chatgpt.site`. Despite the word preview in the hostname, it is the current published Sites deployment. No separate staging environment was verified.

ChatGPT Sites publishes a Cloudflare Worker built with Vinext and Vite. The project uses Next.js App Router-compatible route files, but its main product screens are raw HTML/CSS/JavaScript returned from route handlers. `app/portal.html` is an approximately 155 KB monolithic owner workspace. Standalone account, registration, schedule, scholarship, and planning pages live in route-specific `screen.html` files. React and shadcn-style components are installed, while much of that component tree is starter scaffolding.

Stack verified from package and lock files

Layer	Implementation
Language	TypeScript backend; inline JavaScript in HTML; JavaScript tooling
Framework	Vinext `1.0.0-beta.5`; Next.js dependency `16.3.8`; React/React DOM `19.2.6`
Build	Vite `8.0.13`; Cloudflare Vite plugin `1.37.1`; Wrangler `4.92.0`
Data access	Cloudflare D1 with raw prepared SQL; Drizzle ORM `0.45.2` and Drizzle Kit `0.31.10` for schema/migration definitions
Identity	Supabase Auth through `@supabase/ssr` `0.12.7` and `@supabase/supabase-js` `2.117.3`
Payments	Stripe SDK `23.0.0`, API version `2026-09-30.endive`; test account only
QR	Bundled jsQR `1.4.0`, camera/image scanning, local sample-pass generation
Runtime	Cloudflare Workers; local Node observed at `24.22.1`; package minimum `22.13.0`
Package manager	pnpm `11.25.0`, committed lockfile and supply-chain policy
Styling	Hand-authored CSS for core pages; responsive shared stylesheet; Tailwind/shadcn dependencies also present
Tests	18 Node scripts using assertions, VM-transpiled TypeScript, and isolated `node:sqlite` databases

Request and data flow

Browser requests reach a Sites-managed Worker. Vinext dispatches them to the route handlers under `app/`. Public calendar endpoints project only approved public fields from the owner settings blob. Private handlers validate the Supabase session, derive the role and account ID on the server, and then query D1. The principal account role is owner, coach, family, or anonymous.

Supabase Postgres is used by the connected service for authentication. The application does not store academy business tables in Supabase Postgres and does not use Supabase `.from()` or RPC calls in its

business code. The academy data belongs to Cloudflare D1. No R2 bucket is bound in the hosting manifest.

The Worker can contact Stripe to verify the expected test account when saving a restricted test key. The code has no Checkout creation route or signed payment-webhook handler. A disabled Stripe webhook endpoint exists externally, targeting a route that is absent from this source snapshot.

Three data surfaces

Surface	Storage and trust boundary	Consequence
Owner workspace	One revisioned a `cademy_state.payload` JSON document plus server-managed planning tables	Programs and schedules are published from here; sample kids, purchases, rosters, direct messages, attendance, and payouts also live in this blob
Real family and staff services	Per-user `family_accounts`, coach openings, approvals/chat, curricula/plans, scholarship tables	Authenticated ownership checks govern these records; they are not automatically synchronized into the sample admin directory
Public demonstrations	Fictional data in the browser tab; network-blocking CSP on demos	Useful for interaction review, but not an operational roster or financial ledger

Authentication and authorization

`lib/account-auth.ts` creates the Supabase SSR client and validates each session with `getUser()`. Cookies are HttpOnly, Secure, and SameSite=Lax. Private responses use no-store headers. Mutations generally require same-origin JSON requests.

Owner authorization is hard-coded to the verified email `savagewrestlingclub@gmail.com` and requires AAL2 with a verified TOTP factor. Coaches are granted through `staff_access` and also require AAL2. Families use their verified Supabase user ID, and optional family MFA is enforced after a factor is enrolled. General admin and reviewer roles are not implemented. The sample Admin role shown in the workspace is not equivalent to a backend permission grant.

ChatGPT sign-in helpers and connector infrastructure remain in the starter. They are not a bypass into the owner workspace. Core account data uses Supabase authentication rather than trusting a browser-supplied user ID or ChatGPT display identity.

Service observations

Sites reports version 70 deployed successfully with the same source commit as this package. Runtime environment revision 3 lists `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`, `STRIPE_CONFIG_ENCRYPTION_KEY`, and `STRIPE_TEST_WEBHOOK_SECRET`. Secret values were not retrieved or included.

The connected Supabase project is Savage Wrestling Academy LLC, ref `mdxyylvdgntkjfuiovxi`, region `us-east-1`, status `ACTIVE_HEALTHY`, with a Postgres 17 service. This is a control-plane health observation, not verification of email delivery, recovery, RLS configuration, or business data stored in D1.

Stripe exposes Savage Wrestling Academy LLC in test mode. The nine active test prices match three program products, each with $200 monthly, $40 walk-in, and $2,400 upfront prices. The monthly Price objects themselves do not enforce a 12-payment stop. Subscription scheduling and paid entitlements remain to be built.

AI development versus AI product features

The current development workflow used ChatGPT Work/Codex, Sites, terminal tools, connected Supabase and Stripe tools, and browser-based QA. The repository contains no implemented LLM client, model prompt pipeline, AI assistant feature, RAG index, or vector-search business workflow. Exact AI model versions and the full provenance of earlier generated code cannot be reconstructed from the Git commit messages.

The Sites source/deployment helpers and connector runtime live outside this repository as platform tooling. The application source, migration SQL, bundled QR libraries, build integration, and lockfile are included.

Feature inventory

Statuses describe the current implementation. Implemented plus isolated tests means a code path and assertions exist. It does not mean a production end-to-end transaction was verified. Mocked workspace records can persist to the owner JSON and still remain simulations.

Feature	Status	Actual behavior and limit	Source evidence
Public calendar and program cards	Implemented; public UI verified	Reads saved semester practices for all programs, dates off, named practices and upcoming events. Uses New York dates and month names.	lib/public-schedule.ts; app/calendar/
Season scheduling and holiday exclusions	Implemented; isolated tests	Owner edits weekly dates/times, future semesters and exclusions in the revisioned settings blob.	app/portal.html saveSeason; lib/registration-catalog.ts
Named paid practices / Friday Night Live	Implemented review; payment absent	Owner selects exact dates, title, grades, description and member/non-member prices. Each session is a separate purchase.	lib/friday-live.ts; app/api/booking-review/
Grade-range controls for added practices/events	Implemented display guidance	From/through grade selectors are saved as a range. They do not enforce eligibility at event/practice booking.	app/portal.html offeringGradeFields; lib/friday-live.ts
Upcoming overview	Implemented; public demo verified	Next six items come from saved schedules, named practices and events. Excludes expired sessions and days off. Other-device changes require reloading saved state.	app/portal.html upcomingItems
Email/password signup and login	Implemented; mocked-service tests	Supabase auth routes and session cookies exist. Existing password login and email callbacks are separate paths. Full live email lifecycle was not exercised.	app/api/auth/; app/auth/confirm/
Owner/coach MFA and optional family MFA	Implemented; isolated tests	TOTP enrollment, challenge, verified-factor checks and role destinations are server enforced. No owner bypass through ChatGPT identity.	app/api/security/; lib/account-auth.ts

Feature	Status	Actual behavior and limit	Source evidence
Family profiles	Implemented; isolated tests	Parents save name, phone, wrestler IDs/names/grades. This schema does not yet hold actual waiver documents or USA Wrestling evidence.	app/api/family/; lib/family-records.ts
Admin family directory	Mocked workspace records	Displays sample families/kids from academy JSON. It is not a directory of the real family_accounts table.	app/portal.html familyDirectory
Wrestler/child portal and invitations	Mocked workspace records	Child-access status, messages and QR passes are simulations; no independent child-account backend exists.	app/portal.html childAccessCard; childQr
Paid membership choices	Implemented review; checkout absent	Walk-in, 12 monthly installments for a 12-month commitment, or one upfront annual payment. Totals are server calculated.	app/register/; app/api/registration/
Program grade gates and SWA Advance approvals	Implemented; isolated tests	Real registration uses K-6 / 7-12 rules and approval for every SWA Advance wrestler. Identity/grade edits invalidate stale approvals.	lib/program-approval.ts; app/api/owner-approvals/
Private approval conversation	Implemented; isolated tests	Family/owner-only text thread with role checks, pagination, rate limits and 15-second browser polling. No email/SMS notification.	app/api/approval-chat/; public/approval-chat.js
Team discount codes	Implemented review; isolated tests	Owner codes apply to eligible installment plans only. The backend rejects inactive, wrong-program, upfront and walk-in codes.	lib/academy-validation.ts; app/api/registration/
Paid-member discounts	Partial / not connected	No paid-subscription source of truth exists. General paid membership eligibility cannot yet be verified for Friday Night Live or camps.	app/api/booking-review/

Feature	Status	Actual behavior and limit	Source evidence
Scholarship member discounts	Implemented review; isolated tests	An active real scholarship gives the member practice rate and 10% off Camp reviews. This does not collect money or reserve a place.	lib/scholarships.ts; app/api/booking-review/
Stripe restricted test-key storage	Implemented boundary; live key state unverified	Owner/MFA-only entry, expected-account check, restricted test-key validation and AES-GCM encrypted storage. No live keys accepted.	app/api/payment-setup/; lib/stripe-test.ts
Checkout and payment processing	Not implemented	No session creation, signed fulfillment, orders, subscriptions, invoices, live charge flow, refunds or payment-failure automation.	app/api/payment-setup/ returns checkoutEnabled false
Membership cards and purchase history	Partial / mixed	Real scholarship cards exist. Paid memberships/history remain empty states plus clearly labeled examples.	app/family/screen.html
Scholarship offer management	Implemented; isolated tests	Owner creates email-bound private invitation, expires/rotates/revokes it, freezes tuition reference value, finalizes terms and ends future assistance.	app/api/scholarships/
Scholarship acceptance and free entitlement	Implemented; isolated tests	Invited verified parent selects saved named wrestler. Draft/stale terms cannot be accepted; grade/approval checks and one-active-award uniqueness apply.	app/api/scholarship/
Printed name, drawn signature and date	Implemented; sample UI verified	Explicit electronic consent, bounded signature vectors, exact terms version/hash and server signing time are saved atomically with acceptance.	lib/scholarship-signatures.ts; app/scholarship/

Feature	Status	Actual behavior and limit	Source evidence
Signed scholarship copy	Implemented; isolated tests	Owner or signing family can view, download HTML, or print/save PDF. Earlier typed-name acceptances are labeled as legacy. No automatic email copy.	app/scholarship-agreement/
Scholarship benefit history	Implemented informational calculation	Completed anniversary months since acceptance times frozen tuition reference; partial month excluded; value stops at award end. It is not an invoice.	lib/scholarships.ts benefitSummary
Scholarship repayment / exclusivity enforcement	Draft / not implemented	Proposed cumulative repayment is counsel-review text only. No automatic debt, retrospective invoice, collection or breach adjudication.	lib/scholarships.ts scholarshipDraft
Camps, clinics and event management	Implemented settings; partial registration	Owner events persist and project to public listings; camps can have multiple dates and single-day prices. Real booking stops at review.	app/portal.html saveEvent; app/api/booking-review/
Sample event purchase modal	Confirmed broken path	finishEvent references an undefined annual variable. The isolated reproduction throws ReferenceError, despite the 18-script suite passing.	app/portal.html finishEvent; docs/reproductions/
Capacity and waitlists	Mocked / partial	UI tracks sample slots and offers; review rejects an event with zero slots. No transactional paid capacity or actual waitlist invitation delivery.	app/portal.html waitOffer; app/api/booking-review/
Attendance date/name roster	Mocked workspace records	Opens to today in New York and filters sample kids by program/name. Manual attendance persists only in owner JSON.	app/portal.html attendance; renderRoster
Camera / image QR scanner	Functional sample scanner	Camera and uploaded images are decoded locally; sample passes only. Successful sample scan shows a confirmation and tab-local roster badge.	public/attendance-demo.js; app/attendance-demo/

Feature	Status	Actual behavior and limit	Source evidence
Check-in review popup	Mocked eligibility	Sample missing registration, waiver or USAW status blocks sample check-in. No operational eligibility engine or official USAW lookup.	app/portal.html checkinIssues
Program transfers	Mocked workspace records	Updates sample kid placement and history from today, preserves earlier roster program, and asks for an Advance approval checkbox. Does not move real enrollment or Stripe records.	app/portal.html transferProgram; rosterProgram
Waivers and USA Wrestling cards	Conceptual / sample fields	Actual family profile schema omits waiver/card evidence. Preview can mark sample waiver/status; no secure upload or external verification.	lib/family-records.ts; app/portal.html reviewCard
Coach accounts and openings	Implemented; isolated tests	Owner grants coach permissions. Coach manages only own non-overlapping, exactly one-hour openings and reads published assigned practice plans.	app/api/staff/; app/api/coach/
Private lesson inquiry	Implemented review only	Saved wrestler and lesson format are checked; coach, price and time still need confirmation. No reservation or charge.	app/api/booking-review/ lesson branch
Shared lessons / invitation / 70-30 payouts	Mocked workspace records	One-on-one, partner and group simulations, share links, participant counts, split arithmetic and sample coach payout buttons. No transfer of funds.	app/portal.html lessonSplit; shareLesson; simulateCoachPayout
Sample owner availability calendar	Mocked / separate store	Workspace availability UI is independent of real coach_openings and public booking. Its records need reconciliation before operational use.	app/portal.html availabilitySlots; app/api/coach/

Feature	Status	Actual behavior and limit	Source evidence
Practice plans	Implemented; isolated tests	Owner drafts/shares a scheduled practice plan, up to 90 minutes. Coaches read only published plans in assigned programs; print/save is available.	app/api/practice-plans/; lib/practice-plans.ts
Program curricula and reusable lessons	Implemented; isolated tests	Owner-only program groups and ordered lessons with unique names, revisions, and explicit copy into daily plan.	app/api/practice-curricula/; app/curricula/
General announcements and direct messages	Mocked workspace records	Messages save to owner JSON; family/child role views are simulations. Email and SMS providers are absent.	app/portal.html messages; sendDirect; saveFamilyContact
Event/message archives	Implemented settings/UI behavior	Ended events and related announcements appear archived; manual archive/restore preserves history. Evaluation runs on render/focus/timer, not an external scheduled job.	app/portal.html isEventArchived; refreshArchiveDay
Encrypted backup / restore	Implemented but incomplete coverage	Owner/MFA checks, browser encryption, revision fingerprint and atomic restore/recovery snapshot. Covers only academy state, family profiles and coach openings.	lib/backup-records.ts; public/security-backup.js
Security log	Implemented basic log	Some sensitive actions append security_events for owner review. Coverage is not comprehensive for every mutation or scholarship lifecycle event.	lib/security-events.ts; app/security-log/
Payroll, timesheets, full accounting	Conceptual	No complete operating payroll, tax, settlement or timesheet subsystem is present. Sample lesson split arithmetic is not accounting.	Repository/API/schema inventory

Feature	Status	Actual behavior and limit	Source evidence
Public website / custom domain replacement	Partial	Public calendar/program/booking surfaces exist. Full marketing website migration, custom-domain routing and retirement of the existing website were not verified.	app/calendar/; hosting metadata

Real family directory exception

The owner-only `/accounts` page lists up to 500 real `family_accounts` profiles with saved parent contact fields and wrestler names/grades. This is implemented backend read access, distinct from the fictional Families directory inside the main workspace. It was verified in source, not opened with a live owner session.

Codebase overview

Main source areas

Path	Role
`app/route.ts`	Owner/MFA gate for the main workspace
`app/portal.html`	Owner administration and sample parent/wrestler views, local UI state, save queue, scheduling, sample attendance/commerce/messages, transfer history
`app/*/route.ts` and `screen.html`	Standalone server-rendered raw HTML pages for real account and workflow services
`app/api/*/route.ts`	Auth, settings, profiles, scheduling projections, registration reviews, approvals/chat, plans/curricula, staff, scholarships and backups
`lib/account-auth.ts`, `access-control.ts`, `protected-page.ts`	Session validation, owner identity, role lookups, private page guards
`lib/registration-catalog.ts`, `public-schedule.ts`, `friday-live.ts`, `program-names.ts`	Public projections, prices, program naming, dated offerings and semester schedule generation
`lib/program-approval.ts`, `academy-validation.ts`	Grade/approval rules, team discounts and validation of the settings document
`lib/scholarships.ts`, `scholarship-signatures.ts`, `scholarship-agreement.ts`	Invitation/award data, value calculation, draft terms, signature validation and signed-copy rendering
`lib/practice-plans.ts`, `backup-records.ts`, `stripe-test.ts`, `rate-limit.ts`, `security-events.ts`	Shared backend constraints and security/integration utilities
`db/schema.ts`, `drizzle/`	15 D1 tables, Drizzle metadata, and 13 ordered schema migrations
`public/`	Approval chat, camera scanner, backup crypto, QR encoder/decoder and shared CSS
`config/stripe-test-catalog.json`	Test product/price IDs and intended term metadata; no credentials
`components/`, `hooks/`, `app/globals.css`	Largely starter React/shadcn scaffolding, not the main raw-HTML UI architecture
`build/`, `vendor/`, `lib/connector-*`	Tracked Sites/Vinext Worker integration and connector transport scaffolding
`scripts/`, `vite.config.ts`, `next.config.ts`	Execution profiles, install/build wrappers and local Cloudflare bindings
`.openai/hosting.json`	Existing Sites project identity and DB D1 binding; R2 is null
`tests/`	Node assertion/VM/SQLite tests; no package-level test script or verified CI pipeline

Important state representations

`academy_state` uses one key, academy, with a JSON payload and revision. The payload holds program metadata, semester schedules, named paid practices, events, and team codes. It also holds preview-only kids, bookings, attendance, family directory entries, announcements, direct messages, sample access grants, availability and payout information. One stale global revision rejects the whole save; unsaved UI changes remain local until a reload or successful retry.

`family_accounts` stores a separate JSON profile per verified Supabase user. Each wrestler has an opaque stable ID, name and grade. This is the real family namespace used by registration review, approvals and scholarships. It is not linked to the owner sample kid array by a normalized athlete table.

`scholarship_awards` is a real entitlement record. Its accepted terms and signature snapshot are retained when the owner ends an award. An active scholarship can block duplicate paid registration review and can qualify a wrestler for member review pricing. No corresponding paid-membership table exists yet.

Code-reading order

Read `lib/account-auth.ts`, `lib/access-control.ts`, `app/api/academy/route.ts`, `lib/registration-catalog.ts`, `app/api/registration/route.ts`, then `app/api/scholarship/route.ts`. Follow those into `db/schema.ts`. Finally inspect the owner workspace, because its user-visible breadth is much larger than its operational backend coverage.

Inline JavaScript in HTML is not typechecked by the normal TypeScript run. The confirmed finishEvent error demonstrates why passing the build and TypeScript check is insufficient for the sample admin flows.

The small `app/accounts/route.ts` page is a real owner-only read of family_accounts, capped at 500 profiles. It is separate from the sample family directory in portal.html.

User workflows

Account creation, login and recovery

The login page sends JSON actions to `/api/auth`. Signup validates email and password length, applies rate limits, calls Supabase signUp, and requests email confirmation. A confirmation callback exchanges the code or verifies the token hash, sets cookies, and redirects to login. Login verifies the password in Supabase, derives a role from the verified email or current staff record, and routes to owner, coach or family pages. Owner/coach and enrolled family MFA flows go through `/security` and `/api/security`.

Password recovery calls Supabase resetPasswordForEmail and later updateUser. Both the signup and recovery email destination use the old hard-coded APP_ORIGIN, so verify and correct that configuration before live recovery testing. Existing password login does not exercise the email callback path.

Saving a family

`/family` loads `/api/family`. The server derives the account ID from the Supabase session. The parent saves contact fields and one or more wrestler records. PUT validates the data and expected revision, then inserts or updates only that account. Conflicting changes return 409. A wrestler with an active scholarship cannot be removed while that entitlement remains active. Saving a profile does not purchase a membership or sign a waiver.

Paid membership or walk-in review

The public calendar links into `/register`, with optional program, plan and date parameters. The page loads the authenticated family, a server-generated catalog and approval state. The parent chooses a saved wrestler and one of three plans. Monthly means 12 installments over 12 months; annual means a one-time upfront 12-month term. Walk-ins require a specific saved date/time. Legacy monthly query links now select the installment plan.

POST `/api/registration` validates session ownership, grade, the SWA Advance approval, current program price, any team code, and selected practice. It blocks a duplicate same-program review when a real active scholarship already covers the wrestler. It returns a review with amount, payment count, term, total commitment and draft terms. The response always marks payments disabled. No paid enrollment, invoice, capacity hold, subscription or receipt is created.

Approval and conversation

A parent requests approval for a saved wrestler through `/api/program-approvals`. The owner reviews at `/approvals` through `/api/owner-approvals`. Server checks bind the request to the parent, wrestler ID, name and grade, and owner decisions reject stale revisions. Approval chat is a separate authenticated D1 conversation. Browser polling refreshes it every 15 seconds while visible. Approval permits registration review; it does not create paid access.

Owner scheduling and public calendar

Owner settings load from `/api/academy`. Creating a semester or named practice changes the in-memory owner document and queues a revisioned PUT. A successful save updates D1; public projections subsequently read that state. Daily practice generation chooses the latest matching semester for each program and removes excluded days. Public calendar views always show all programs. Named practices have selected dates and separate prices. Event definitions include dates, type, price, optional single-day camp price, description, grade guidance and slot count.

Up next is built from saved owner-state schedules and upcoming offerings. This fixes the prior fixed October examples, but it is still a view of the current loaded workspace and is not a realtime subscription to another device's changes.

Camps, clinics, added practices and lessons

`/booking` submits a saved wrestler and offering selection to `/api/booking-review`. The server verifies the live offering and date. A real active scholarship qualifies for the saved member practice price and a 10% Camp review discount. Paid member status is not available yet. A full event is blocked from review, but no transactional reservation or payment is made. Private lesson review verifies a format and remains an inquiry; it does not assign a coach or reserve a time.

Owner workspace event purchases, shared lessons, payout buttons and waitlist offers are simulations. The sample camp/event purchase modal currently throws a ReferenceError. The real `/booking` API path is separate and passed its isolated tests.

Scholarship offer, agreement and signature

The owner opens `/scholarships`, enters a wrestler name, parent email and program, then creates an offer through `/api/scholarships`. The server snapshots the saved monthly tuition as an informational reference value and creates a random 256-bit invitation token. Only its SHA-256 hash is stored. The returned private link places the token in the URL fragment. It is copied manually; no email or SMS is sent. Invitations expire after 30 days, can be rotated, and are bound to the invited verified parent email.

The initial agreement is draft text with acceptance closed. The owner must paste final attorney-reviewed terms and affirm review before the server marks the offer ready. This is an owner declaration, not an external legal-validation service. The default draft's proposed cumulative repayment does not itself create debt or trigger collection.

The parent opens the link, authenticates with the invited email, and selects the saved matching wrestler. The parent enters a printed name, draws a signature, sees the date, and checks agreement and electronic-signature consent. The server revalidates grade/approval, token, expiry and current terms revision. It atomically saves an active free scholarship and an immutable signature record with vectors, signer, account email, terms hash/version and server timestamp. A conditional database update also protects against concurrent family-profile or approval changes. A uniqueness constraint prevents two active awards for the same account/wrestler.

Both owner and family can access `/scholarship-agreement?id=...` after authorization. It returns the accepted terms, signature and signing record; an HTML attachment can be downloaded and browser printing can save PDF. Signature vectors are excluded from bulk award-list responses. Earlier typed-name records remain printable and are explicitly labeled as legacy. No automatic email copy or certified third-party e-sign service is integrated.

Ending a scholarship changes future entitlement while retaining its accepted agreement. Informational tuition value counts completed anniversary months since acceptance at the frozen rate and stops at the

award end. Earlier unconditional training is not backdated into that calculation. No repayment amount is automatically invoiced or charged.

Attendance and transfers

The owner Attendance view opens to today's New York date. Name search filters preview kids. Manual check-in evaluates sample registration/waiver/USAW fields and writes an attendance flag to the owner JSON. The camera scanner decodes sample passes, shows confirmation, and posts a same-origin message to the owner page. A tab-local roster badge appears once. Sample scans do not create real D1 attendance rows.

Transfer program changes a sample kid's program from today and records placement history for earlier roster dates. Its Advance approval checkbox does not create a real program_approvals decision. This transfer does not modify real family IDs, scholarship program assignments, paid subscriptions or payment amounts.

Coach, planning and curricula

The owner grants real coach access through `/staff`. A verified coach signs in with MFA and edits only their own one-hour openings. The database rejects overlapping openings atomically. Paid booking is not connected to those openings.

Practice plans are keyed to program/date. Owner changes require a current scheduled practice, a valid revision and a plan totaling at most 90 minutes. Owners save drafts or publish. Active assigned coaches read only currently scheduled, published plans for their programs. Curricula are owner-only program groups containing ordered lessons; applying a lesson copies it into a daily draft without automatically publishing or coupling future edits.

Messaging, archives and backups

Approval chat is a real family-scoped message service. General direct conversations, announcements, SMS consent controls and child messaging in the workspace are sample JSON records. There is no email/SMS delivery provider connected to those flows.

Event and related announcement archiving is derived from dates or explicit archive flags. Refresh occurs through render, focus/visibility and a minute timer, rather than a server cron job.

The owner backup tool exports only academy state, family profiles and coach openings. The browser encrypts the export with AES-GCM and a passphrase-derived PBKDF2 key. Restore validates the project and fingerprint, creates a recovery snapshot, and writes atomically. Missing backup records are retained and existing revisions increase. Scholarship contracts/signatures, approvals/chat, plans, curricula, staff permissions and integration secrets are not included by this export. Provider-level backups and actual production recovery were not verified.

Database schema and migrations

The application business database is Cloudflare D1, a SQLite-compatible database bound as DB. Supabase owns authentication users and factors in a separate service. No application Supabase business tables were identified.

The review applied all 13 original SQL migrations, in order, to an empty isolated SQLite database and exported the resulting 15-table schema. docs/schema.sql contains the actual resulting CREATE TABLE and index statements. docs/schema.json contains column and index metadata. Neither contains production records. This verifies migration replay on an empty SQLite database, not the applied migration version of the live D1 instance.

Table	Purpose
academy_state	One revisioned JSON academy configuration and sample workspace state.
family_accounts	Real verified family profile JSON keyed by Supabase user ID. Separate from sample directory.
auth_attempts	Rate-limit counters keyed by identity/request bucket.
staff_access	Owner-managed coach access keyed by email, assigned programs stored in text.
coach_openings	Real coach availability windows.
security_events	Selected security/admin events, not comprehensive accounting audit.
recovery_snapshots	Limited legacy backup payloads in the same database.
restore_guard	Constraint-based restore validation support.
program_approvals	Per-family wrestler and program approval status.
approval_messages	Owner/parent thread messages for a program approval.
practice_plans	Per-program/date JSON practice plans, published flag and revision.
practice_curricula	Ordered lesson templates in curriculum groups, JSON payload and revision.
practice_curriculum_groups	Per-program named curriculum grouping.
integration_secrets	Encrypted restricted Stripe test credential, wrapping key outside DB.
scholarship_awards	Offer, recipient, frozen terms, status, acceptance and signature evidence.

Relationships and constraints

Supabase user IDs are stored as text in family_accounts, coach_openings, approvals and scholarship records. These references are application-managed across services. Wrestler IDs live inside family JSON and are referenced by approvals and awards. Sample kid IDs inside academy_state are a separate identity space.

Unique indexes enforce one approval per family/wrestler/program, one plan per program/date, distinct curriculum names within programs, distinct lesson names within a curriculum, unique invitation hashes,

and one active scholarship per family/wrestler. CHECK constraints bound approval states, message roles, publication flags, scholarship states/readiness and restore validation. The schema does not provide a comprehensive foreign-key network or relational athlete/enrollment/payment model.

JSON payloads keep flexible UI state but move validation, relationship integrity, migration and concurrency responsibilities into application code. Several updates use optimistic revision matching. The single academy JSON document remains a contention and growth boundary.

No canonical paid memberships, Stripe event ledger, invoices, payment orders, durable capacity reservations, actual QR passes, real attendance ledger or waiver-evidence tables exist. These are dependencies for a production enrollment system.

Migration lineage

- `0000_jazzy_silvermane.sql`
- `0001_harsh_ser_duncan.sql`
- `0002_funny_james_howlett.sql`
- `0003_lonely_valeria_richards.sql`
- `0004_fair_sentinels.sql`
- `0005_gigantic_blazing_skull.sql`
- `0006_brief_nomad.sql`
- `0007_spooky_reptil.sql`
- `0008_lumpy_nemesis.sql`
- `0009_faithful_bulldozer.sql`
- `0010_futuristic_vulcan.sql`
- `0011_nebulous_shinobi_shaw.sql`
- `0012_brainy_runaways.sql`

Drizzle's schema definitions live in db/schema.ts, with generated snapshots and journal in drizzle/meta. Inspect both the SQL history and current schema before generating a new migration. Do not overwrite past migrations or run the combined schema export on an existing populated database.

Route reference

This inventory is extracted from the reviewed app/api route files. Mutations generally use JSON, same-origin validation and the route-specific account/MFA guard. This is a review map, not a replacement for request schemas and action unions in each route. Read the route and its lib helper before writing a client. Never infer authorization from a hidden navigation link.

Route	Methods	Purpose and access
/api/academy	GET, PUT	Owner-only shared academy configuration/sample operational state, optimistic revision writes.
/api/approval-chat	GET, POST	Owner or matching family thread participants. Bounded messages, pagination and rate limiting.
/api/auth	GET, POST	Supabase identity/session status and sign-in, signup, resend, reset, password and sign-out actions. Action-specific guards and rate limits.
/api/backups	GET, POST	Owner encrypted-export/restore support for the limited legacy dataset. Does not cover all tables.
/api/booking-review	POST	Verified family event/lesson/Friday Live review. No booking, payment or reservation.
/api/coach	GET, POST	Active MFA coach identity, assigned programs, availability openings.
/api/family	GET, PUT	Verified account profile read/update with account scoping and revision checks.
/api/owner-approvals	GET, POST	Owner decisions on pending program approvals.
/api/payment-setup	GET, POST	Owner reads status or validates and encrypts restricted Stripe test key. Checkout remains disabled.
/api/practice-curricula	GET, POST	Owner manages curriculum groups and ordered lessons.
/api/practice-plans	GET, POST	Owner authors and publishes, assigned coach reads eligible published plans.
/api/program-approvals	GET, POST	Family-scoped Advance approval requests and status.
/api/public-schedule	GET	Public sanitized monthly projection. No-store and cross-origin read allowed.
/api/registration	GET, POST	Public configuration plus authenticated wrestler/program registration review. Payments disabled.
/api/scholarship	GET, POST	Token-based offer review, matching verified recipient acceptance with frozen terms and signature.
/api/scholarships	GET, POST	Owner lists, creates, revises, rotates, revokes and ends awards.
/api/security	GET, POST	Session/MFA enrollment and challenge actions. Not a public security-log API.
/api/staff	GET, POST	Owner-only active coach assignments by email.

Additional server-rendered pages include /accounts, /account-setup, /approvals, /backups, /curricula, /practice-plans, /payment-setup, /security-log, /security and /staff. Each protected page performs its own identity check. /auth/confirm completes the Supabase email flow. /scholarship-agreement returns an authorized printable accepted-agreement representation. It is not a public contract download.

The source contains no /api/stripe-webhook route and no checkout-session creation route. The registered Stripe webhook target does not imply a working handler. Most route implementations contain action dispatch inside POST rather than separate REST resource paths. API errors, ownership checks and accepted body shapes are directly testable in tests/*.mjs.

Setup and continued development

What was validated

The reviewed source was exported to a separate directory, with a symlink to the existing installed dependency tree. All 18 test scripts, a non-incremental TypeScript check, and a portable-profile production build passed there. No clean dependency installation, alternate operating-system install, production database migration replay, real signup email, MFA challenge or payment was executed during this review.

A new development checkout

Use the packaged `source/` directory or obtain access to the canonical Sites source repository from Justin. The snapshot has no Git object directory and no remote configuration. Preserve the reported base commit and hashes if importing it into GitHub. Create a separate development branch/environment before any implementation work.

The package requires Node >=22.13.0; Node 24.22.1 was used for this review. Prefer matching Node 24 and pnpm 11.25.0 initially, because the tests use node:sqlite and the project has version-specific build tooling. The package manifest and lockfile are authoritative for dependency versions.

```
node --version

pnpm --version

pnpm install --frozen-lockfile
```

A fresh installation was not attempted here. The local dependency audit reached the package registry successfully. Recheck the lockfile and platform build policies on the collaborator's machine.

The original README is stale. Its description of npm ci and Bash-free portability conflicts with the current `install:ci` entrypoint, which runs `bash scripts/install-pnpm.sh` and requires tools such as flock and timeout. A normal pnpm install is the practical starting point for a portable checkout; verify that it reproduces the lockfile before relying on it. Do not translate this established project to npm casually.

Local configuration

Copy the package's `.env.example` into the new source checkout as `.env.local` and replace the placeholders with credentials for a separate development Supabase project. The Cloudflare Vite plugin supports .env or .dev.vars for local Worker env bindings. The existing .gitignore excludes .env files, but not .dev.vars, so do not put secrets in a tracked .dev.vars file. No production secret is included in this package.

Current application inputs are SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY and, for Stripe credential storage, STRIPE_CONFIG_ENCRYPTION_KEY. The encryption key is 32 random bytes encoded as 64 hexadecimal characters. Keep the real wrapping key with the encrypted credential; changing it invalidates stored encrypted keys unless they are re-encrypted deliberately.

STRIPE_TEST_WEBHOOK_SECRET is configured in the hosted environment, but the current application does not read it. It does not enable webhooks or checkout. The restricted test API key is encrypted in D1 integration_secrets through the owner page, not supplied by a source file or a STRIPE_SECRET_KEY environment variable.

APP_ORIGIN and OWNER_EMAIL are source constants in lib/account-auth.ts. An environment variable of those names will not override them. The former currently points at the older wellrose34 hostname. A

developer needs to make an intentional code/configuration change in their own branch and align the Supabase Site URL and redirect allowlist before testing local signup and recovery. The portable ChatGPT mock identity does not bypass Supabase owner authentication.

Official local-env reference, checked during review, https://developers.cloudflare.com/workers/local-development/environment-variables/

Local schema and build

A clean exported checkout defaults to the portable profile, because the ignored execution-profile selection is absent. The build in this state passed with the existing dependencies. The tracked build integration simulates a local D1 binding named DB. There is no persistent root wrangler.jsonc in this project; the build emits dist/server/wrangler.json.

```
pnpm run build
```

For a new empty local D1 database, apply all SQL files under drizzle in filename order. Use the project's generated config and the same persistence directory. Apply each migration once; do not replay a migration already applied to that local database.

```
node --import ./scripts/sites-env.mjs ./node_modules/wrangler/bin/wrangler.js d1 execute DB --local --config
dist/server/wrangler.json --persist-to .wrangler/state --file drizzle/0000_jazzy_silvermane.sql
```

Repeat the command with each next migration, ending at 0012_brainy_runaways.sql. The schema-only combined export in docs/schema.sql is useful for inspection or a new isolated schema, but the application migration history remains the 13 original files and Drizzle journal. Do not apply both paths to the same database.

```
pnpm run dev
```

In a portable checkout the development server starts on port 5173 unless changed. For a built local Worker, `pnpm start` uses Wrangler on loopback and .wrangler/state. Use the URL printed by the process. Secure-cookie and callback behavior on local development still needs end-to-end verification with the selected Supabase configuration.

Tests and audit

There is no npm test script. Run each tracked test directly from the source root, or use the included non-mutating runner from this package.

```
node ../tools/run-tests.mjs

node node_modules/typescript/bin/tsc --noEmit --incremental false

pnpm run build

pnpm audit --prod --json
```

The runner invokes the 18 scripts sequentially. They mostly use Node VM mocks and empty SQLite databases populated from migrations. They do not prove production email delivery, actual payments, browser camera compatibility, a full migration from the current site, or end-to-end customer operation.

The audit returned eight advisories in this review. Do not run audit fix blindly; Vinext compatibility and the actual deployed dependency graph require assessment. A full dependency and built-artifact audit is still needed, because some runtime-relevant framework packages are declared as devDependencies.

Existing Sites development and publishing

Inside ChatGPT Work, the Sites integration owns source-repository credentials, source synchronization, version saving, production environment bindings and deployment. Its site-workflow, package and deployment helpers are platform tooling outside this repository. They were used during prior development, but no source credential was minted and no deployment was performed during this review.

The existing .openai/hosting.json already identifies this Site. Do not create a replacement Site accidentally. Reusing its project ID publishes to the current live application. For collaboration, obtain the appropriate Sites editor/source access from Justin and use an isolated environment until the changes are reviewed. Owner access inside the application is a separate Supabase/MFA permission.

For a standalone Cloudflare deployment, supply new Worker/D1 infrastructure and adapt the Sites dispatch/connector integration and secrets. That portability path has not been verified. Do not assume a plain next start, Vercel deployment, or GoDaddy upload is equivalent to the current Worker build.

Version control and provenance

The inspected original checkout is clean and its HEAD is bf444860637d6fc94fbd0342bf7ba0963ed0f178. It is not shallow and contains 73 local commits. Sites version 70 records the same commit; the associated deployment reports succeeded. This verifies alignment of the inspected committed source with the latest published version's provenance.

`git remote -v` and the remote URL configuration query returned no configured remotes in this checkout. No GitHub repository URL, GitHub Actions workflow, or verified GitHub mirror was found. The code is committed locally and its deployed Sites version has matching source provenance. A current complete copy committed to GitHub cannot be confirmed from the available evidence.

The package's source folder is a Git archive of all 252 tracked files at HEAD. It deliberately preserves the original source README and SECURITY document rather than silently rewriting application repository files during a read-only review. New handoff documentation lives at the package root and in docs. Git history is not bundled; historical commits were not scanned for secrets. Justin must provide canonical repository access or authorize a separate secure history export if history is needed.

Included and excluded dependencies

Item	Included	Continuation requirement
App source, tests, config, build integration, vendor files, public assets	Yes, exact tracked snapshot	Verify source hashes against docs/source-manifest.json
pnpm lockfile, workspace policy and package manifest	Yes	Reinstall dependencies with the pinned package manager
node_modules and native platform binaries	No	Clean installation on the target machine remains unverified
.sites-runtime execution profile and cache	No	Clean clones default to portable; managed Sites helpers select the managed profile
.wrangler and database contents	No	Provision a separate local D1 DB and apply migrations; production data access must be arranged separately
Supabase users, sessions, MFA factors and email settings	No	Owner-managed Supabase configuration and access
Hosted environment secret values and encrypted Stripe credential	No	Secure owner-controlled provisioning; preserve wrapping-key recovery
Sites source credentials and publication helpers	No	Supplied by the Sites platform, outside the application repository
Live tenant settings, schedules, family/award records and signatures	No private data dump	Public observations are documented; the runtime data store is not the source repository
Generated dist/server deployment output and prior tar archives	No	Rebuild from source; the isolated build passed
Domain registrar/GoDaddy configuration and other prior design files	No	Not verified or needed for the current source build; coordinate separately for migration

No common hard-coded secret pattern was found in the current tracked snapshot. This heuristic scan is limited to selected private-key, Stripe-secret, Supabase-secret, GitHub-token and JWT patterns. It is not a guarantee that the repository, dependencies, history or service configuration are free of secrets or vulnerabilities.

Source history uses broad Update Site source commits, so feature-level authorship and exact AI provenance are not fully reconstructible from commit subjects. There is no verified automated CI/CD, branch protection, release approval process or collaborator access policy in GitHub.

Known issues and technical debt

Priority P0 blocks a production launch. P1 is high business or operational risk. P2 improves maintainability or requires targeted validation. Confirmed findings are separated from architectural limitations and unverified concerns. No remediation was applied during this review.

SEC-01 Dependency advisories

Priority P0. Evidence status Confirmed package audit.

Eight production-graph advisories, including two critical, affect declared Next.js, Zod and fast-uri versions. Runtime exploitability is unproven because this deployment uses Vinext rather than a standard Next server.

Proposed work. Review evidence/dependency_audit.json, trace built-artifact reachability, upgrade or remove affected packages in a development branch, and regression-test the exact Worker build.

AUTH-01 Old authentication callback origin

Priority P0. Evidence status Confirmed source/config mismatch.

APP_ORIGIN points at the old wellrose34 Site while current deployment is under justinaccordino546. Signup/resend/reset emails reuse the constant; account-setup displays it too. Live redirect behavior and Supabase allowlist were not tested.

Proposed work. Align configurable allowed origins and actual Supabase settings. Test confirmation, recovery, password changes, logout and MFA recovery with controlled accounts.

DATA-01 Two wrestler/membership namespaces

Priority P0. Evidence status Confirmed architecture gap.

academy_state.kids drives the admin sample directory, roster and transfers, while family_accounts and scholarship_awards drive real family workflows. No canonical paid enrollment or operational athlete table bridges them.

Proposed work. Design one stable identity/enrollment model, migrate sample-vs-real separation explicitly, and make roster/access derive from real entitlements.

DATA-02 Incomplete backup coverage

Priority P0. Evidence status Confirmed source limitation.

Manual exports omit scholarships, signed terms and signature records, approvals/chat, plans/curricula and other normalized tables. Recovery snapshots are in the same D1 instance.

Proposed work. Create a complete protected backup/recovery path and retention policy, preserve secret wrapping keys separately, and prove restoration into a separate environment.

PAY-01 Checkout and signed fulfillment absent

Priority P0. Evidence status Confirmed source and Stripe metadata.

No Checkout creation or webhook handler exists. Stripe endpoint targeting /api/stripe-webhook is disabled, and payment APIs explicitly return disabled state.

Proposed work. Build and test order/checkout lifecycle, authentic signed webhooks, idempotency, paid entitlement and failure/cancellation/refund reconciliation before any live charge.

DEMO-01 Sample event review throws ReferenceError

Priority P1. Evidence status Reproduced in isolated VM.

finishEvent renders annual?0:base-price without defining annual. It affects the sample camp/event review modal. Existing 18 test scripts and TypeScript/build do not detect the inline-JS failure.

Proposed work. Run the packaged reproduction. Fix the scoped expression and add a regression covering the sample event entry point. Do not confuse the separate working booking-review API with this modal.

DEMO-02 Sample grade and approval rules drift

Priority P1. Evidence status Confirmed source discrepancy.

bookDrop still uses Development grades 1-5, Intermediate 6-12 and only a junior Advance approval check. Real registration correctly enforces K-6, 7-12 and all Advance grades. editKid also lacks Kindergarten.

Proposed work. Consolidate grade/approval representations across both interfaces and test every normal entry point, or clearly limit/remove stale sample registration paths.

SEC-02 Compromised-password protection disabled

Priority P1. Evidence status Supabase advisor confirmed.

The connected Supabase project reports auth_leaked_password_protection as WARN.

Proposed work. Review the project plan/configuration and enable the supported protection. Remediation https://supabase.com/docs/guides/auth/password-security#password-strength-and-leaked-password-protection

AUTH-02 Single hard-coded owner and incomplete recovery operations

Priority P1. Evidence status Confirmed design; operations unverified.

Only one verified owner email is recognized. There is no real admin/reviewer role, owner handover UI or verified emergency MFA recovery process.

Proposed work. Define least-privilege collaboration and recovery procedures. Test owner/coach recovery without weakening the existing AAL2 requirement.

BOOK-01 Capacity and lesson booking are not transactional

Priority P1. Evidence status Confirmed implementation limit.

Real APIs review choices only. Sample slot counts, waitlist offers and lesson participation/payouts are not a live reservation ledger; real coach openings are disconnected from paid booking.

Proposed work. Implement atomic inventory/holds and coach assignment with retry/expiry rules, then payment fulfillment and cancellation release behavior.

ATT-01 Real QR attendance and participation evidence absent

Priority P1. Evidence status Confirmed implementation limit.

Scanner is sample-only; actual family schema lacks waiver/card evidence and there is no real check-in table.

Proposed work. Build operational entitlement/waiver/card checks, secure pass issuance, revocation and idempotent per-session attendance records before using it at the door.

MSG-01 General messaging has no delivery service

Priority P1. Evidence status Confirmed implementation limit.

Approval chat works as a protected stored thread, but general direct/announcement/email/SMS flows save sample owner JSON only.

Proposed work. Define consent and delivery requirements, integrate a transactional channel, and add receipts/failure handling and scoped family conversations.

SCH-01 Scholarship contract and collection remain policy work

Priority P1. Evidence status Confirmed implementation boundary.

Attorney review is represented by an owner checkbox and pasted terms. There is no external legal validation or automatic repayment billing. Benefit totals exclude the current partial month and never backdate earlier unconditional support.

Proposed work. Finalize the agreement and legal review outside the code, clarify benefits/termination/exceptions and records retention, and keep financial collection separate from informational value.

SCH-02 Signature record is not externally certified

Priority P1. Evidence status Confirmed design limitation.

Record uses authenticated account identity, saved vectors/consent/time and a SHA-256 of terms. It has no trusted timestamp provider, external signing certificate, or independent tamper-evident record store.

Proposed work. Decide required evidentiary assurances with counsel and implement appropriate durable audit/export controls. Preserve legacy acceptance records accurately.

ARCH-01 Large inline-JS owner workspace

Priority P1. Evidence status Confirmed maintainability gap.

About 155 KB of raw HTML includes many UI and business functions. TypeScript does not analyze those inline scripts. Owner payload validation is broad for some nested sample records.

Proposed work. Extract shared typed domain logic and modules incrementally; preserve callers and existing saved state. Strengthen relevant validation and add missing entry-point tests.

ARCH-02 Whole-document saves and broad list queries

Priority P2. Evidence status Confirmed design; no load test.

Every owner edit rewrites a revisioned JSON blob, capped at one million request characters. Some lists use fixed 500/1000/5000 limits without UI pagination. User counts and peak concurrency were not measured.

Proposed work. Normalize transactional records, paginate bounded endpoints, evaluate query plans and load-test realistic club usage. Retain optimistic conflict protection.

OPS-01 No verified independent staging, monitoring or full recovery drill

Priority P1. Evidence status Unverified / incomplete source evidence.

Current published URL is live; no isolated staging, alerting, incident runbook, retention schedule or independent production recovery test was verified.

Proposed work. Provision separate test services and credentials, add monitoring, backups, operational ownership and launch acceptance checks.

DOC-01 Original docs and setup wrappers are stale

Priority P2. Evidence status Confirmed discrepancies.

README retains starter instructions and old pricing notes; SECURITY narrows Advance approval to grades 7-8. install:ci invokes a Linux Bash wrapper despite older portability claims.

Proposed work. Adopt the handoff docs into the main repository only after review, verify a fresh portable install, and align scripts with the documented package manager.

DEMO-03 Admin-demo form sandbox needs browser review

Priority P2. Evidence status Static-review concern, not reproduced.

The admin-demo CSP enables scripts/modals but omits allow-forms, while several demos use form submit handlers. The signing sample explicitly allows forms and blocks network requests.

Proposed work. Exercise admin-demo form submissions in supported browsers and confirm desired sandbox behavior. Keep real owner APIs inaccessible from the public demo.

SEC-03 Security validation scope is incomplete

Priority P2. Evidence status Unverified / static risks.

No penetration test, full dependency/dev graph review or production load test was performed. Some handlers enforce body size after reading text, and public/invite endpoint abuse controls need assessment.

Proposed work. Review body limits, rate limiting, CSP, logging/PII retention and abuse scenarios before accepting production data at scale.

Prioritized development roadmap

Complexity is relative, not a time or cost estimate. S is a bounded fix, M spans several components, and L changes the transactional model or external integrations. Dependencies matter more than the order of the visual menu.

Work	Priority	Complexity	Depends on	Acceptance condition
Triage package advisories and deployed runtime reachability	P0	M	Exact source/build baseline	Patched/removed dependencies or documented non-reachability, full relevant regression and build audit
Correct callback origin and verify email/MFA lifecycle	P0	M	Controlled Supabase dev configuration	Signup, verification, login, recovery, password change, logout and protected roles work on intended origins
Establish isolated staging and least-privilege developer access	P0	M	Owner-controlled service access	Changes and test charges cannot modify the live data environment accidentally
Complete backup and signature retention/recovery	P0	M	D1 schema inventory, wrapping-key custody	All required records recover into a clean environment; accepted contracts survive termination and recovery
Unify athlete, family, enrollment and entitlement identities	P0	L	Data migration design	Owner directory/roster, real family portal, scholarship and paid access use stable linked IDs
Finalize ordinary and scholarship agreements	P0	External policy work	Owner/counsel review	Clear term, pricing, early exit, notices, privacy and signature procedures are approved before binding enrollment
Implement test checkout and immutable order records	P1	L	Unified identity, policy, test credentials	Server-calculated amounts and snapshotted policy/discount choices produce safe retryable test sessions
Implement signed webhook and reconciliation lifecycle	P1	L	Checkout/order model	Valid signatures/account/mode, replay protection, idempotent fulfillment and out-of-order handling are proven
Implement fixed 12-installment and upfront entitlements	P1	L	Stripe lifecycle and canonical memberships	First payment counts toward 12; no 13th charge; end-of-month/leap dates, failures, cancellation and manual continuation are tested
Replace sample billing cards with actual records	P1	M	Orders, subscriptions and receipts	Correct last payment, next installment, term end, failure status and receipts per wrestler
Implement operating roster, waiver/card evidence and QR check-in	P1	L	Real entitlements and secure data handling	An eligible wrestler appears on the right session, a scan records once, and missing requirements produce an actionable staff review

Work	Priority	Complexity	Depends on	Acceptance condition
Connect added practices, camps and capacity	P1	M/L	Orders, entitlements and transactional holds	Correct member pricing, date/grade eligibility, seat limits, holds, cancellations and no duplicate enrollment
Connect private lesson booking to real coach openings	P1	L	Capacity, payments and coach identities	No double booking; actual time/coach assignment and supported shared-participant lifecycle
Implement general scoped messaging and delivery	P1	M/L	Real family directory and consent policy	Sender/recipient isolation, opt-ins, channel delivery and failure visibility are verified
Fix sample event error and grade-rule drift	P1	S/M	Known reproduction and shared domain rules	Both real and demonstration entry points pass regression tests and communicate their actual status
Refactor raw HTML logic and improve entry-point tests	P1/P2	M/L	Behavior inventory and data model	Shared typed rules replace duplicated logic; critical UI workflows run in browser integration tests
Improve observability, audit coverage, pagination and CI	P2	M	Stable services and version-control home	Pull-request checks, deployments, alerts, error ownership and load thresholds are defined and measured
Implement financial reporting / coach settlement if required	P2	L	Verified financial model and reconciliation	Fees, refunds, payouts, receipts and business reporting reconcile to actual transactions
Complete website/domain migration and launch rehearsal	P2	M	Production acceptance of account and transaction flows	Correct links/domain/callbacks, mobile/browser checks, operator training, small-group rehearsal and rollback plan

Payment implementation requirements

Use the existing server grade/approval checks and recompute prices from saved configuration. A recurring Stripe Price alone is insufficient to implement a fixed 12-month commitment. Choose and test a bounded schedule, renewal/termination semantics and invoice accounting before live billing. Handle asynchronous success/failure, duplicate and unordered events, refunds, disputes and fraud warnings. Do not grant paid access from a browser success redirect.

Stripe test mode is the only verified account context. No live charge authorization, live keys or live purchases were part of this review. Before live operation, verify tax obligations, receipts, cancellation rights and all required contract notices with the appropriate business/legal review.

Suggested first collaborator session

Run the shipped tests and the sample event reproduction, read the architecture and feature matrix, compare the real /family and /accounts services with the sample admin directory, and review the canonical identity/entitlement model before committing to a production estimate. Then agree with Justin on the first isolated milestone and its acceptance checks.

Demo and access

The public deployment is https://savage-registration-preview.justinaccordino546.chatgpt.site . Sites version 70 was confirmed deployed successfully from the reviewed commit. The preview hostname is the current live Site. No separate staging environment was verified.

URL path	Purpose and access
/calendar	Public schedule drawn from saved configuration
/admin-demo	Public sample owner workspace, fictional records
/scholarship-signature-preview	Public fictional scholarship signature form
/attendance-demo	Sample camera/pass demonstration, not real attendance
/login	Real Supabase sign-in
/register	Real registration review, payment disabled
/booking	Event and lesson review, no purchase or reserved place
/family	Verified family account, optional enrolled MFA challenge
/	Owner workspace requiring owner identity and MFA
/accounts	Owner-only list of up to 500 real family profiles
/scholarships	Owner offer administration
/scholarship	Recipient invitation flow, valid link and matching verified account required
/coach	Active coach role and MFA required

Use the public sample pages first. The sample parent view shows fictional Alex Demo and Sam Demo. The scholarship signature preview uses fictional data and does not create an accepted agreement. It has no production signing effect.

No shared live test-account credentials were verified or created. The test scripts use mocked identities, passwords and authenticator codes. Those are not usable platform accounts. For an end-to-end review, create a dedicated non-customer account in an isolated development Supabase environment after correcting the origin and redirect configuration. Ask Justin to grant the intended owner/coach access through the existing service controls. Do not borrow his password or MFA seed.

The screenshots in docs/screenshots were captured from public pages during this review. They show the fictional owner dashboard, fictional family overview, sample signature form and actual public December 2026 calendar. The calendar can change after capture. No private family account or accepted signature was captured.

Review did not submit a real registration, change a schedule, create an invitation, sign an agreement, access customer records, send an email, or take a payment. Some live flows therefore remain unverified despite passing isolated tests.

Verification record and limits

Review date November 5, 2026. The reviewed commit is bf444860637d6fc94fbd0342bf7ba0963ed0f178. This review made no application changes, deployment, hosted environment change, account creation, production data mutation or payment.

Evidence obtained

- Exact tracked source archive, 252 files, file sizes and SHA-256 manifest.

- Local Git HEAD, clean working tree, 73 commits, no remotes and non-shallow state.

- Sites version 70 and successful deployment metadata pointing to the same source commit.

- Hosted environment variable names with secret values excluded.

- Supabase project health and security-advisor warning metadata.

- Stripe test account, nine active prices and disabled webhook configuration using read-only APIs.

- All 18 existing test scripts passed in an isolated source copy using the already installed dependencies.

- TypeScript non-incremental check and portable production build passed in that copy.

- All 13 migrations replayed against empty SQLite to derive the 15-table schema.

- Production-dependency audit returned eight advisories. Raw results are in evidence/dependency_audit.json.

- Public demo/calendar/signature pages were inspected in browser and screenshots captured.

- Additional isolated reproduction confirmed finishEvent throws ReferenceError for undefined annual.

Important limits

Tests primarily use VM mocks and isolated SQLite. Passing tests do not establish production email, MFA recovery, payment fulfillment, camera access or browser compatibility. No clean installation or full browser end-to-end suite was run. Existing installed dependencies were linked into the separate validation directory. A full dependency audit including runtime-relevant devDependencies was not completed. Advisory reachability in the deployed Vinext build is unassessed.

Production D1 rows and migration state were not queried. Supabase redirect settings, email provider delivery, full authentication configuration, backups, paid plan, retention and production recovery were not inspected. Actual accepted scholarships, family counts and saved test-key state were not accessed. No live owner or family session was used.

GitHub synchronization, canonical GitHub ownership, external CI/CD, standalone Cloudflare account access and untracked historical assets cannot be confirmed from this checkout. No remote is configured. The Sites deployment provenance matches the inspected commit, which verifies the current deployed source reference but does not prove every external operational dependency is in Git.

A source-pattern scan found no obvious live secret values in tracked files. This is not a full Git-history secret audit. Service-managed secret values were deliberately not retrieved. Source includes non-secret service identifiers and synthetic test data.

Original README and SECURITY.md contain stale statements, documented in the issues list. New documentation in this package supersedes those descriptions for handoff purposes while the archived

source remains unchanged.

Reproduce the additional bug

From the source directory with Node available, run `node ../docs/reproductions/sample-event-reference-error.mjs`. The script reads app/portal.html and runs only an isolated sample VM. It reports the undefined annual variable in finishEvent. It does not request a live endpoint or modify the source.

The existing test result list is evidence/tests.json. Individual stdout files are evidence/test_logs. Typecheck stdout is empty because the command passed without diagnostics. Build stdout and audit output are included separately.

Access required for collaboration

Justin should grant individual collaborator access through each service, with the minimum role needed. Account passwords, MFA seeds, bearer bypass tokens and private keys are not part of this handoff.

Service or resource	Needed access and current uncertainty
ChatGPT Sites project	Editor/source access to the existing Site and its version/deployment controls. Current source is packaged; platform tooling and deployment credentials are external.
GitHub	Confirm whether a canonical repository exists. No configured remote was found. If absent, create a private repository with owner authorization and import reviewed source/history through the canonical source channel.
Supabase	Development project and Auth configuration access. Production project reference mdxyylvdgntkjfuiovxi. Inspect redirects, SMTP/email, MFA policy and leaked-password protection.
D1 / Cloudflare hosting	Verify live database binding, migration state, recovery/export and platform-supported access. A standalone Cloudflare account relationship was not verified.
Stripe	Team access to the Savage test environment first. Confirm restricted key permissions and future live-mode ownership without sharing secret values.
App owner role	The app currently recognizes a hard-coded owner email plus verified MFA. Platform editor access does not grant this role.
Legal/business decisions	Final cancellation, early-exit, renewal, scholarship duration/repayment and guardian terms need authorized review. Existing signature capture does not establish enforceability.
Domain, email delivery, support	Custom domain/DNS ownership, transactional email configuration and operating support procedures remain unverified.

Prior AI tooling and exact model history cannot be reconstructed from files alone. Verified current tooling includes ChatGPT Work, Codex, Sites, terminal tools and service integrations. The project contains no implemented AI model feature in its application runtime. Ask the project owner for additional vendor records or historical context where necessary.

Outstanding tasks

The detailed sequencing, acceptance criteria and complexity estimates are in docs/ROADMAP.md. All items below are open at the end of this read-only review.

- [ ] Establish collaborator access and a canonical current Git repository.

- [ ] Assess and remediate dependency advisories in the actual deployed runtime.

- [ ] Correct APP_ORIGIN and align Supabase redirect/email configuration.

- [ ] Enable/check Supabase leaked-password protection and test MFA/recovery.

- [ ] Fix the sample event ReferenceError and add a meaningful regression check.

- [ ] Align sample grade rules and copy with backend eligibility.

- [ ] Build and verify complete, offsite, restorable backups covering all records.

- [ ] Unify real athletes, enrollments, transfers and operational rosters.

- [ ] Obtain final authorized membership and scholarship terms.

- [ ] Implement test-mode checkout, signed idempotent webhooks, durable billing state and finite-term enforcement.

- [ ] Verify payment failures, cancellations, refunds, renewal/term completion and reconciliation.

- [ ] Implement real attendance, QR passes, waivers and capacity enforcement.

- [ ] Connect messaging, booking and payouts to actual delivery/accounting as required.

- [ ] Add full end-to-end, accessibility, security and restore/load checks.

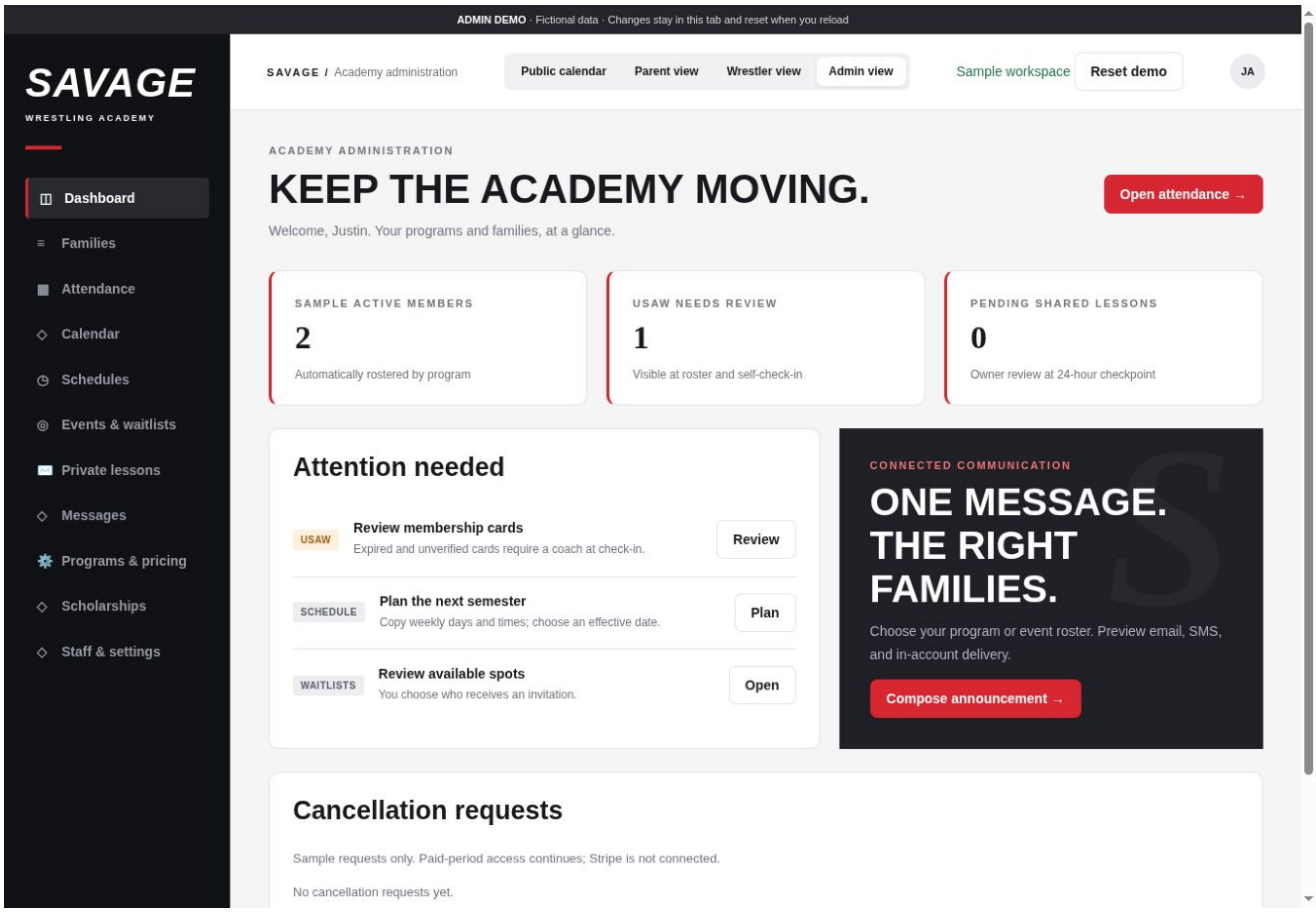
- [ ] Define operational monitoring, support, privacy retention and release procedures.

- [ ] Update source README and SECURITY.md in a separate implementation change after review.

- [ ] Approve a controlled production rollout only after these dependencies are resolved.

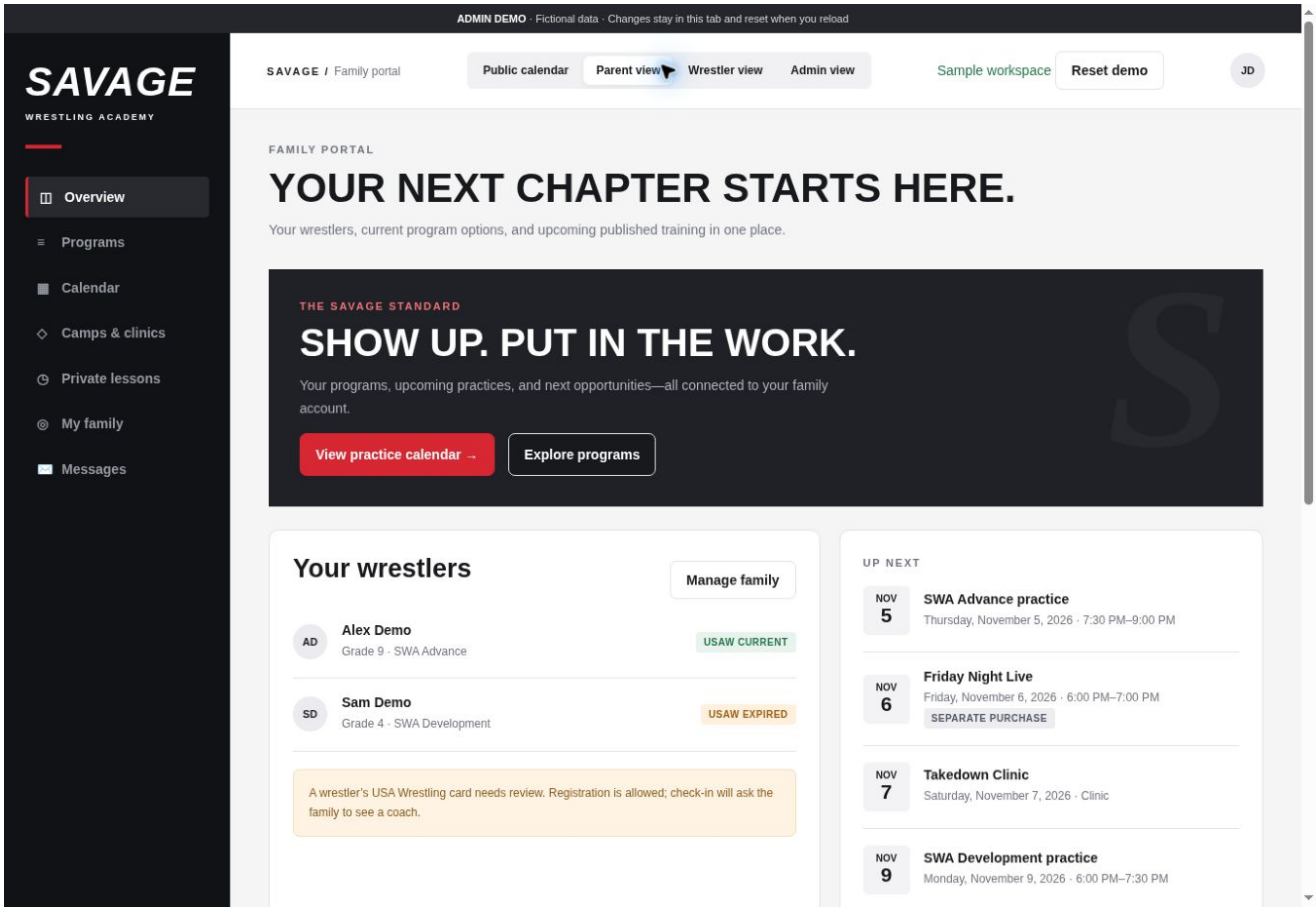
Public-page screenshots

Public sample owner workspace. Fictional data.



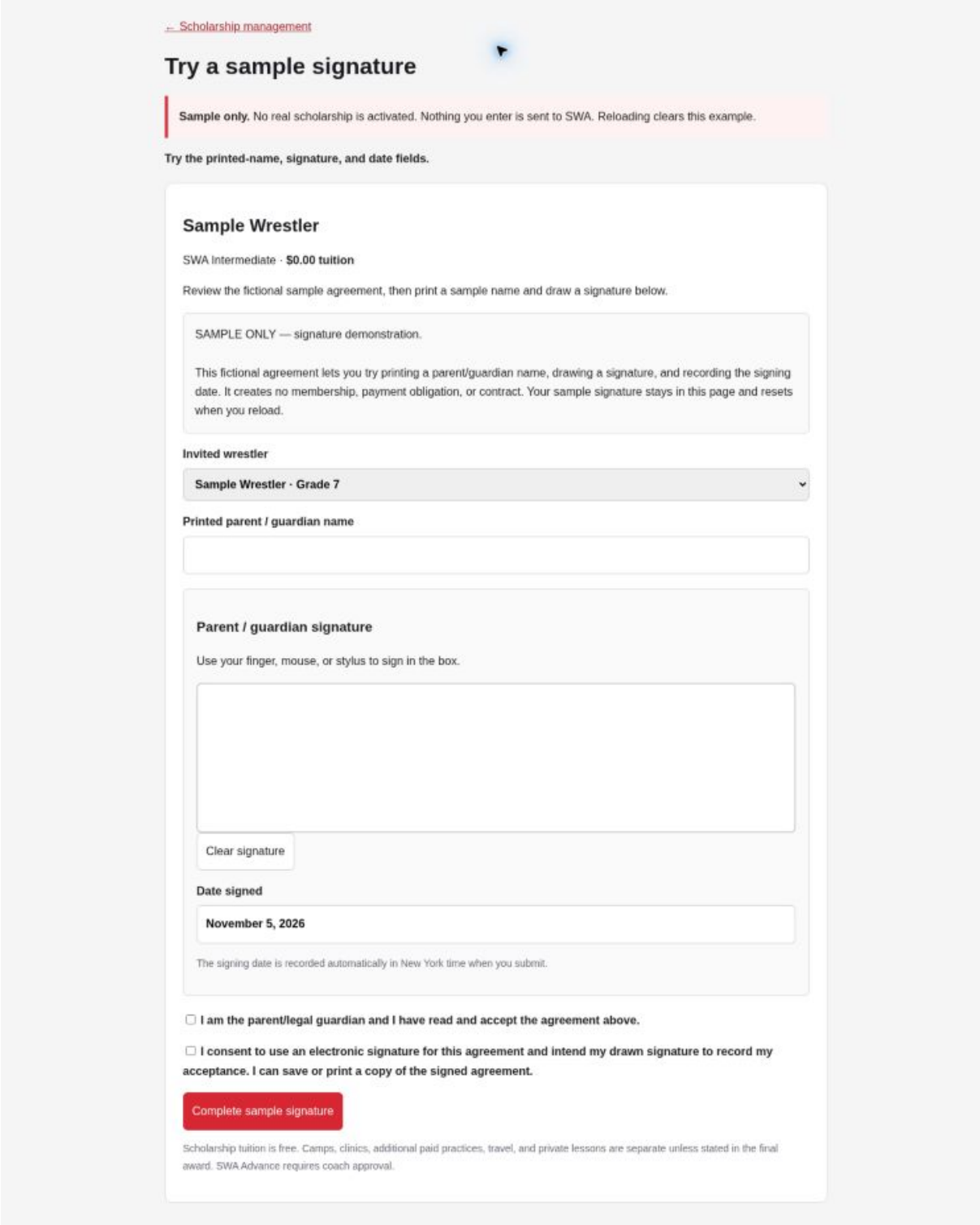
Screenshot file · docs/screenshots/swa_handoff_admin_demo.png

Sample family overview. Fictional data.



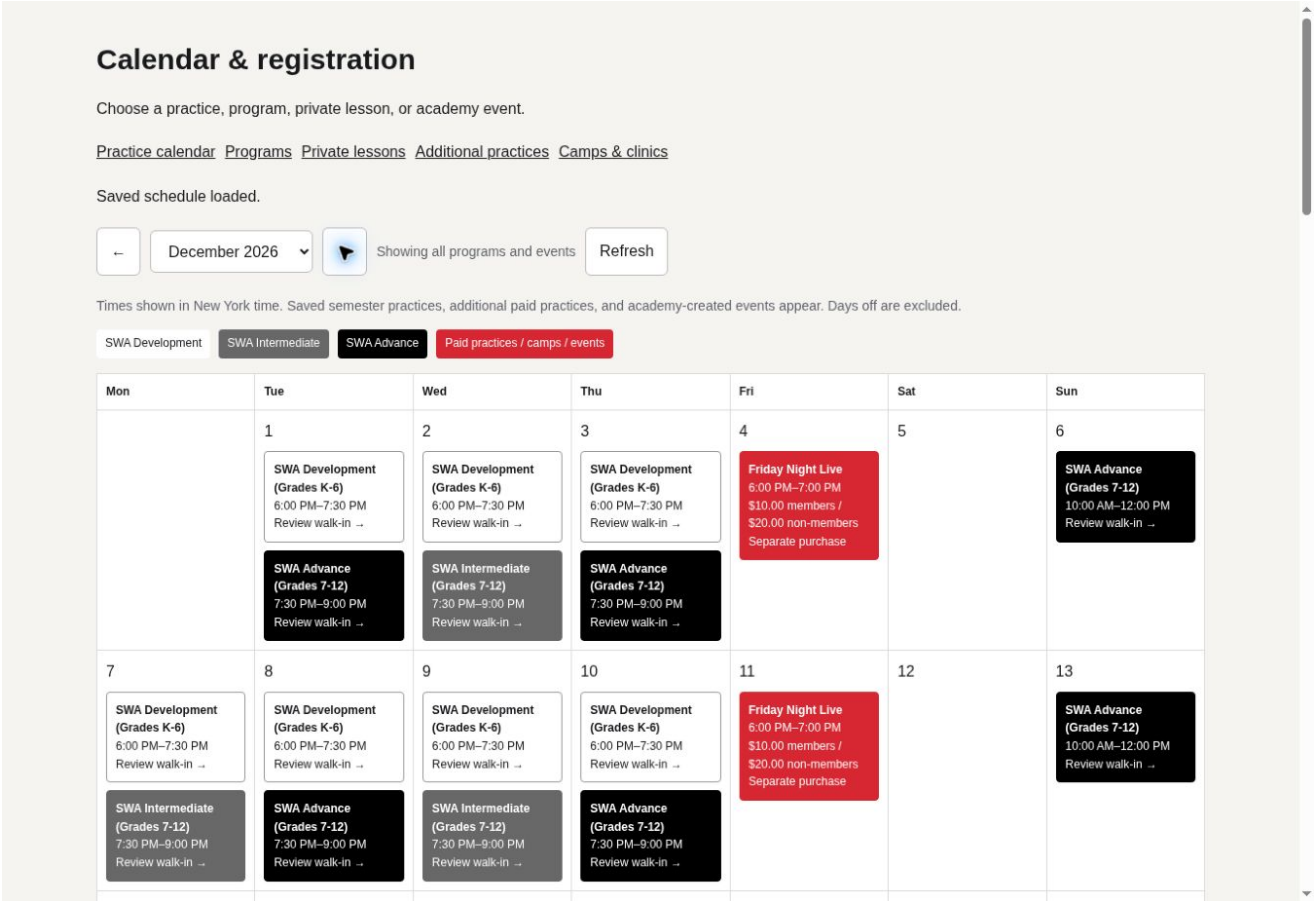
Screenshot file · docs/screenshots/swa_handoff_family_overview.png

Sample scholarship signature form. No real agreement submitted.



Screenshot file · docs/screenshots/swa_handoff_signature_preview.png

Published December 2026 calendar at review time. Mutable configuration.



Screenshot file · docs/screenshots/swa_handoff_public_calendar.png